IAM Attack & Defense Lab

OpenID Connect / OAuth 2.0 - security lab documentation

A hardened OIDC relying party and a deliberately vulnerable twin,
with four reproducible OAuth attacks and the controls that defeat them.

Keycloak 26 - PostgreSQL 16 - FastAPI / Authlib - Docker Compose

LAB ONLY

Deliberately insecure by design. Plain HTTP, placeholder secrets. Never expose or reuse these settings.

INTRODUCTION

1. Overview

This lab demonstrates the same OpenID Connect application built two ways - securely and insecurely - so the effect of each control can be observed directly. A hardened relying party runs against a locked-down realm (**secure-lab**); an identical app configuration runs against a deliberately weakened realm (**vuln-lab**). Four attacks are then run against the weak realm, and each is shown to fail against the hardened one.

The stack is Keycloak 26 as the identity provider, PostgreSQL 16 as its store, and a FastAPI relying party using Authlib for the Authorization Code flow with PKCE. Everything runs locally under Docker Compose.

Architecture

The browser reaches Keycloak as `localhost:8080`; the app, inside Docker, reaches it as `keycloak:8080`. Keycloak is configured so the issuer and login page stay on `localhost` while back-channel endpoints resolve to `keycloak`, keeping the token `iss` claim consistent for validation.

```
Browser  --login (localhost:8080)-->  Keycloak  ---  PostgreSQL
Browser  --app pages (localhost:8000)-->  FastAPI app
App      --code->token, JWKS (keycloak:8080, back-channel)-->  Keycloak
Attacker --malicious redirect (vuln-lab)-->  :6660 listener  --redeem-->  Keycloak
```

SECURE VS. VULNERABLE

2. The two realms

Both realms hold the same users. Every control below is enabled in **secure-lab** and disabled in **vuln-lab**.

Control	secure-lab	vuln-lab
Client type	confidential (secret)	public (no secret)
Redirect URIs	one exact URL	* (any)
PKCE	required (S256)	not enforced
Implicit flow	off	on
Direct access grant	off	on
Brute-force protection	on	off
Self-registration	off	on

Lab credentials (both realms)

User	Password	Roles
alice	alice-password	lab-user
admin-amy	amy-password	lab-user, lab-admin
victim (vuln-lab)	Password1	lab-user
Keycloak admin	admin	-

All credentials are placeholders, printed deliberately, and must never be reused.

SECURE-LAB / DEMO-RP

3. Defense: the secure build

The hardened relying party uses the Authorization Code flow with PKCE (S256) and a confidential client. The implicit and direct-access-grant flows are disabled, the redirect URI is a single exact value, and protected routes enforce roles server-side.

The issuer-mismatch fix

Keycloak runs with a full-URL hostname and dynamic back-channel resolution, so the discovery document returns different hosts for front- and back-channel endpoints:

```
issuer        : http://localhost:8080/realms/secure-lab   (browser-facing)
authorize     : http://localhost:8080/.../auth            (browser-facing)
token_endpoint: http://keycloak:8080/.../token            (app-facing)
jwks_uri      : http://keycloak:8080/.../certs            (app-facing)
```

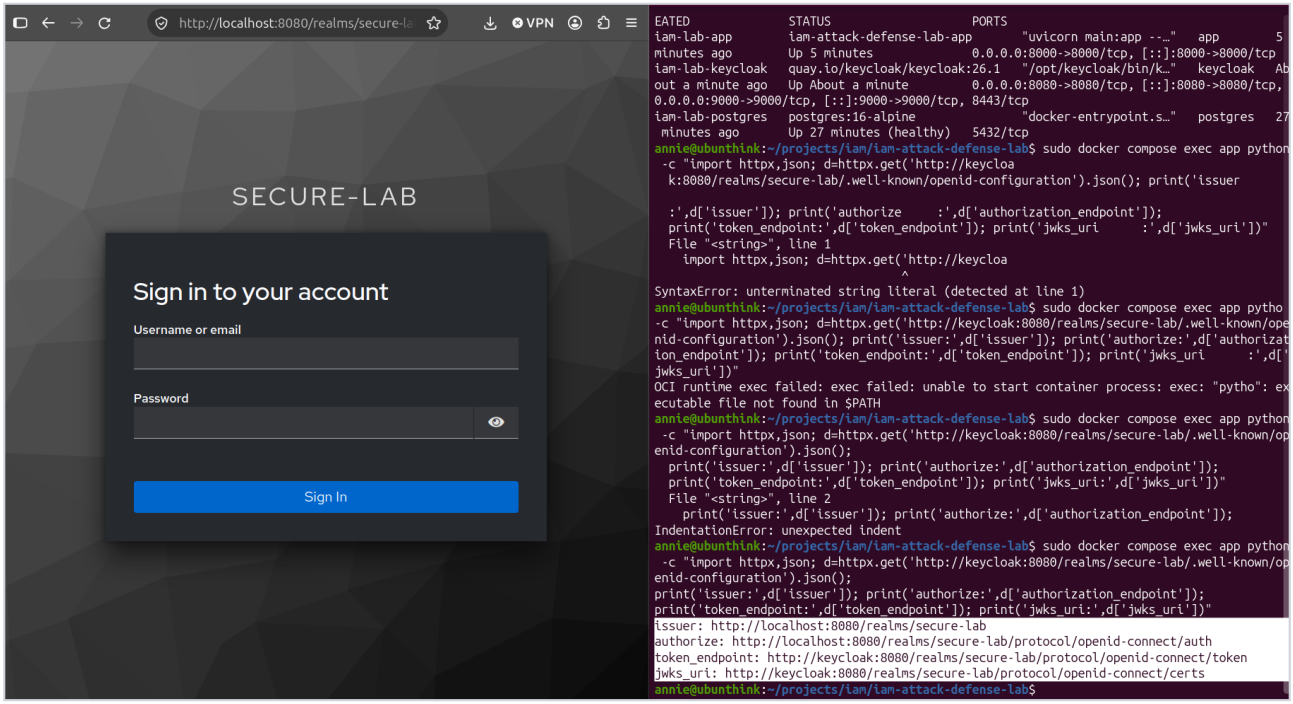


Figure 1. The discovery document as the app sees it: issuer and authorize on localhost, token and certs on keycloak.

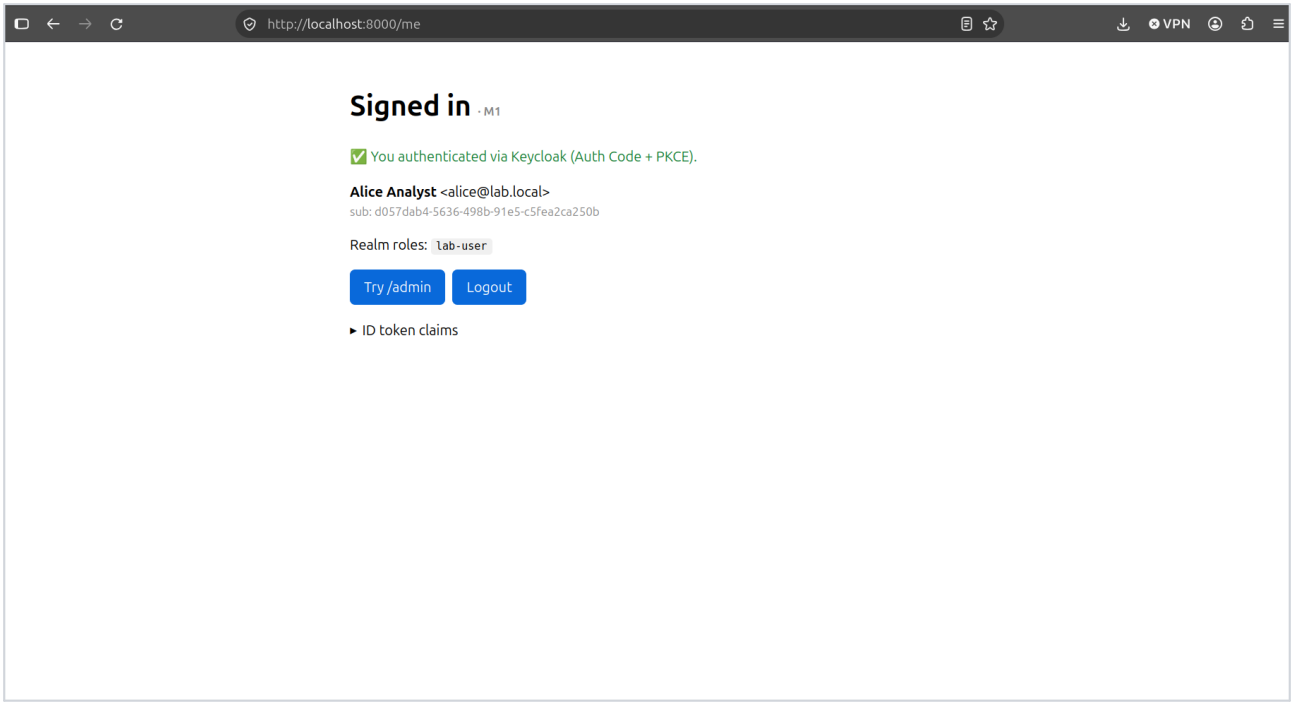


Figure 2. A successful login via Authorization Code + PKCE, showing the user's identity and realm role.

Role-gated access

The `/admin` route requires the `lab-admin` role, checked server-side on every request.

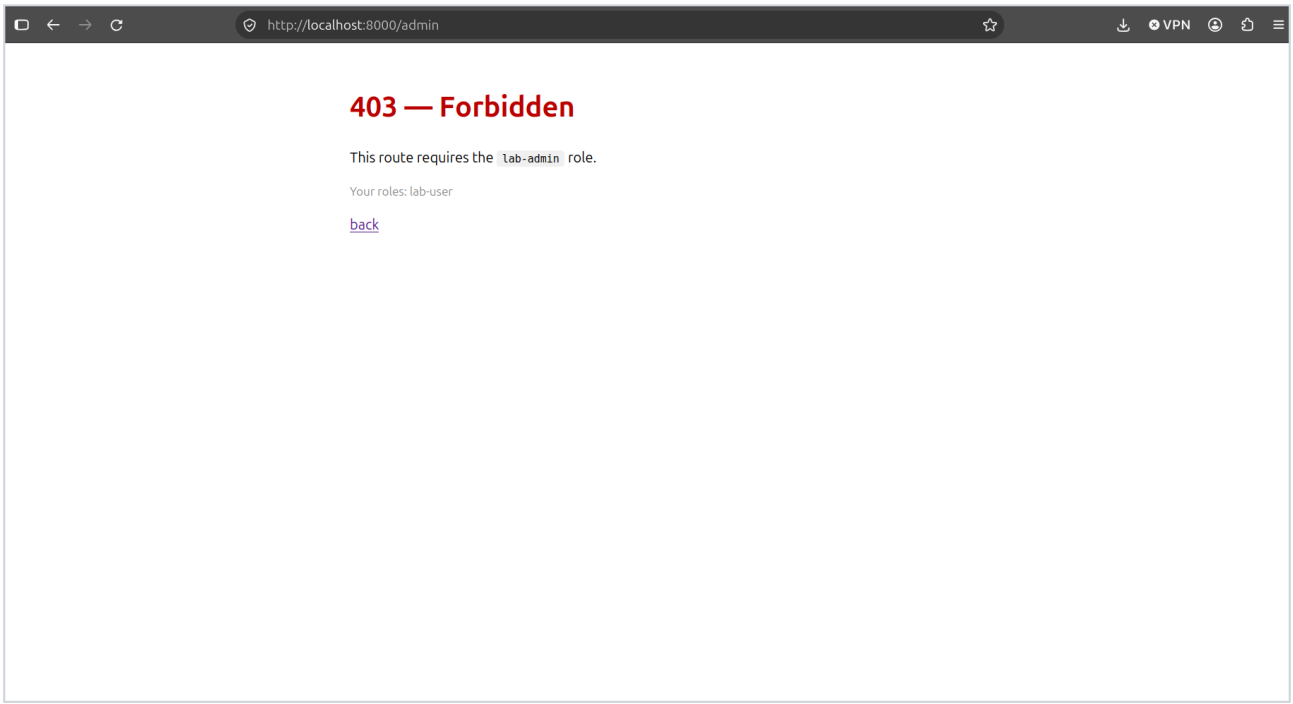


Figure 3. A standard user (alice) is denied access to the admin route.

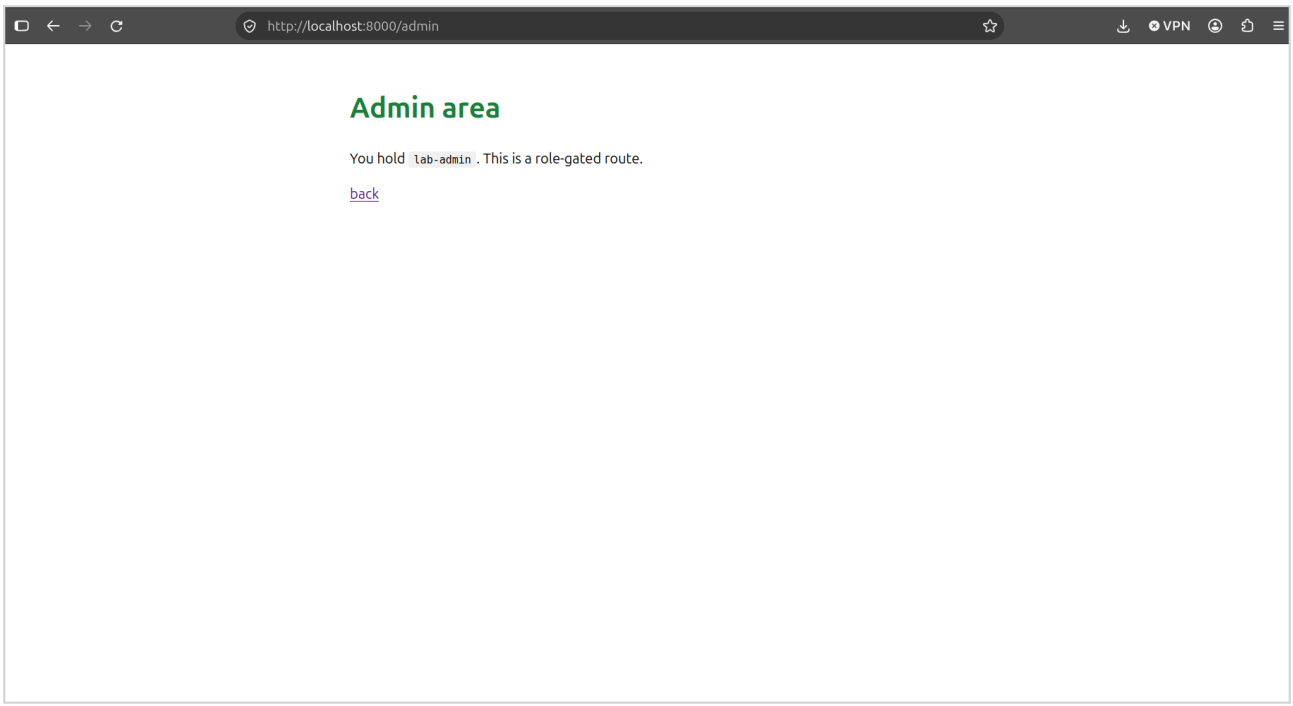


Figure 4. An administrator (admin-amy) is granted access.

VULN-LAB / VULN-RP

4. Attacks

Each attack below targets the weak realm and is contrasted with the secure realm's refusal. Scripts live in `attacks/`.

4.1 Direct Access Grant and brute force

Weakness: the password grant is enabled, so a username and password can be traded for tokens with no browser or app. With brute-force protection off, the token endpoint becomes a password-guessing oracle.
Defense: secure-lab disables the grant and refuses the request.

```
annie@ubunthink:~/projects/iam/iam-attack-defense-lab$ ./attacks/01-direct-access-grant.sh

[1] Password grant against vuln-lab (victim / Password1)
  -> SUCCESS: received an access token.
     sub:    1b48adea-fcf9-4c8e-8ad6-88ff9c82ae58
     user:   victim
     roles:  ['lab-user']

[2] Brute forcing 'victim' (no lockout on vuln-lab)
  [-] tried 123456 -> HTTP 401
  [-] tried password -> HTTP 401
  [-] tried qwerty -> HTTP 401
  [-] tried letmein -> HTTP 401
  [+] FOUND: victim / Password1   (HTTP 200)

[3] Same password grant against secure-lab (defense in place)
  -> REFUSED: unauthorized_client - Invalid client or Invalid client credentials

Done. Fix: disable Direct Access Grants; use Authorization Code + PKCE.
```

Figure 5. Tokens issued for a password on vuln-lab, the password brute-forced, and the same request refused on secure-lab.

4.2 Redirect URI hijack

Weakness: the client accepts any redirect URI, so a login link for the real client can send the victim's authorization code to an attacker server; being a public client with no PKCE, the code is then freely redeemed for tokens. **Defense:** secure-lab pins one exact redirect URI and refuses the malicious link before login.

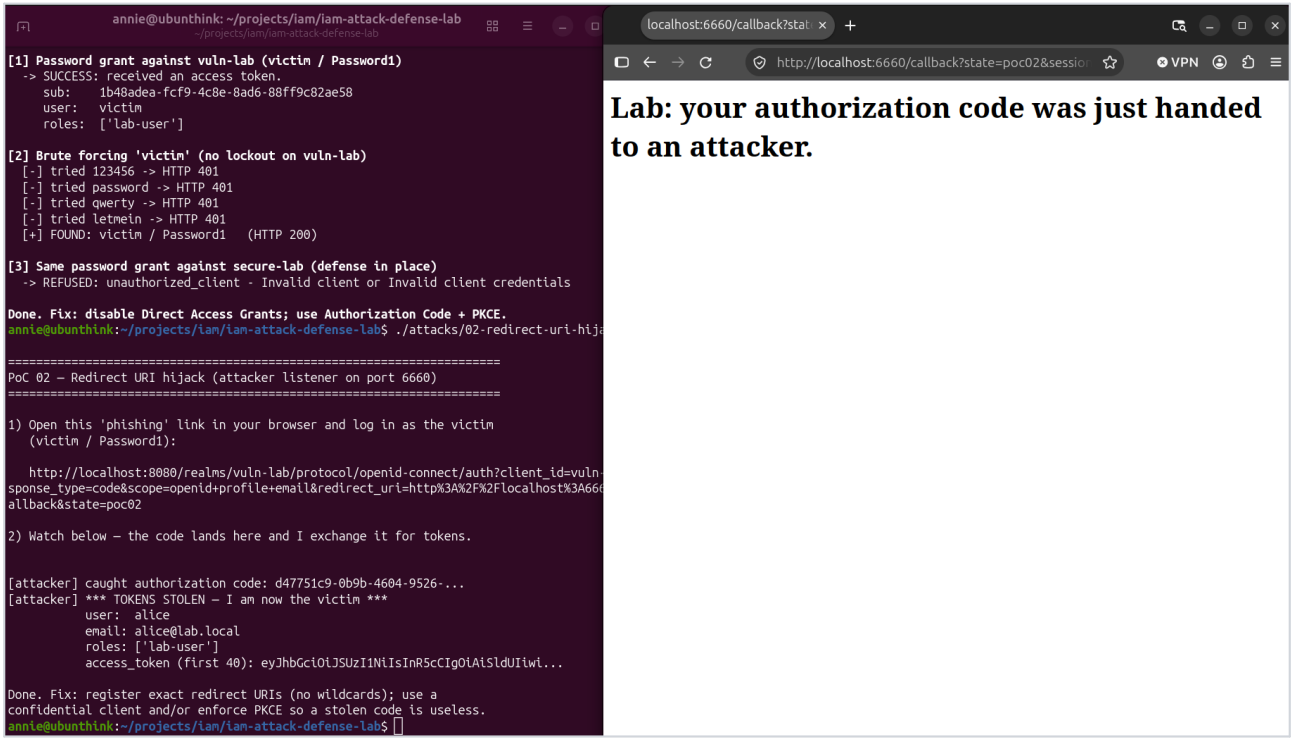


Figure 6. The attacker server captures the victim's authorization code (one click, existing session) and exchanges it for tokens.

4.3 Implicit flow token leak

Weakness: the implicit flow returns the access token in the URL fragment, where page JavaScript, the address bar, and browser history can read it. **Defense:** secure-lab disables the implicit flow entirely.

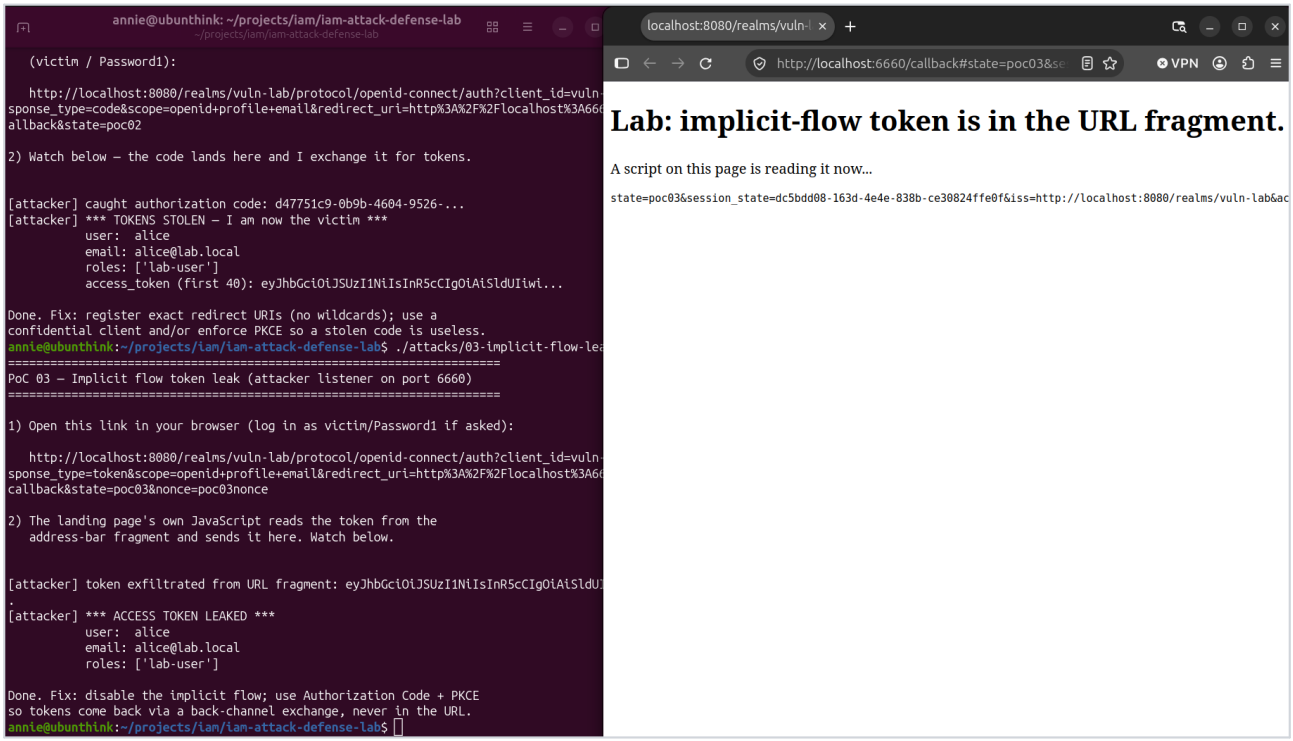


Figure 7. The access token is delivered in the URL fragment and exfiltrated by the landing page's own script.

4.4 Missing PKCE enforcement

Weakness: PKCE is not required, so an authorization code is not bound to the client that started the flow and an intercepted code is redeemable by anyone. **Defense:** secure-lab requires PKCE (S256) and rejects any flow without a challenge.

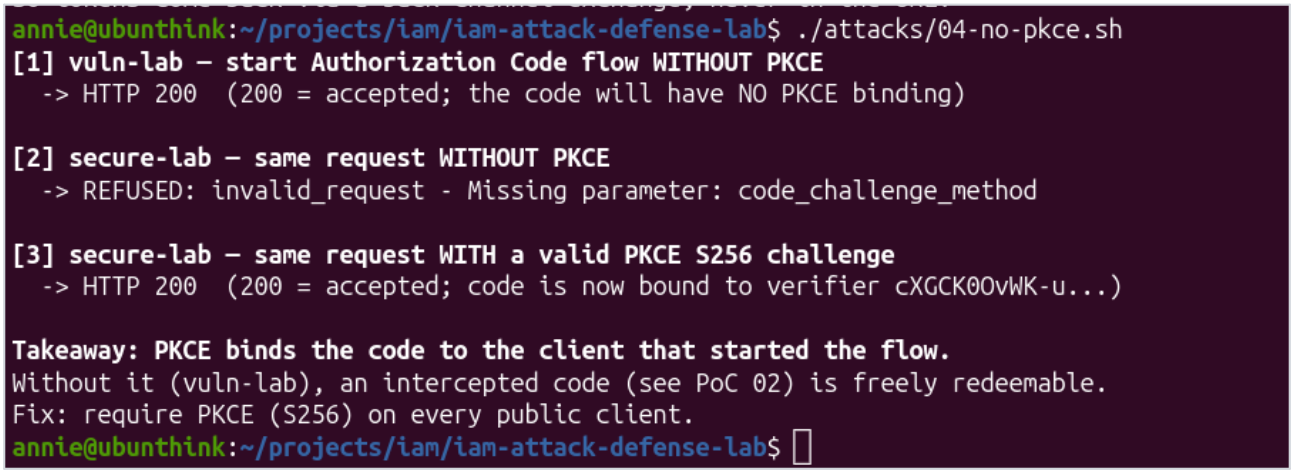


Figure 8. vuln-lab accepts an auth request with no PKCE challenge; secure-lab requires one and accepts a valid S256 challenge.

ANALYSIS

5. Threat model and ATT&CK; mapping

The design principle: secrets and token exchange live on the back channel; nothing sensitive crosses the front channel in a URL. Each vulnerability violates this.

Attack	Weakness	STRIDE	ATT&CK;
Direct access grant	ROPC + no lockout	Spoofing, Elevation	T1110, T1078
Redirect URI hijack	wildcard redirect	Spoofing, Disclosure	T1557, T1539
Implicit token leak	implicit flow on	Info Disclosure	T1539, T1550
Missing PKCE	PKCE not enforced	Tampering, Spoofing	T1557, T1550

CONTROLS

6. Hardening checklist

The baseline that defeats every attack above:

- Use the Authorization Code flow with PKCE (S256); disable implicit and direct-access grants.
- Prefer confidential clients; keep the secret out of source control.
- Register exact redirect URIs - no wildcards, no wildcard hosts or paths.
- Validate every token: signature (JWKS), iss, aud, exp, nonce.
- Short access-token lifetimes; idle and absolute session timeouts.
- Brute-force protection / lockout at the identity provider; MFA for privileged roles.
- Enforce roles and permissions server-side on every protected route.
- TLS everywhere (omitted here only because the lab is local).
- Pin image versions; keep secrets, keys, and tokens out of git.

REFERENCE

Appendix: quick start

```
cp .env.example .env
docker compose up --build -d
docker compose ps

# app:    http://localhost:8000   (Login with Keycloak)
# admin:  http://localhost:8080   (admin / admin)

./attacks/01-direct-access-grant.sh
python3 attacks/02-redirect-uri-hijack.py
python3 attacks/03-implicit-flow-leak.py
./attacks/04-no-pkce.sh
```